

HELIOS2018: RoboCup 2018 Soccer Simulation 2D League Champion

Hidehisa Akiyama[1(✉)], Tomoharu Nakashima[2], Takuya Fukushima[2], Jiarun Zhong[2], Yudai Suzuki[2], and An Ohori[2]

[1] Fukuoka University, Fukuoka, Japan
akym@fukuoka-u.ac.jp
[2] Osaka Prefecture University, Osaka, Japan
tomoharu.nakashima@kis.osakafu-u.ac.jp,
takuya.fukushima@edu.osakafu-u.ac.jp

Abstract. The RoboCup Soccer Simulation 2D League is the oldest of the RoboCup competitions. The 2D soccer simulator enables to teams of simulated autonomous agents to play a game of soccer with realistic rules and sophisticated game play. This paper introduces the RoboCup 2018 Soccer Simulation 2D League champion team, HELIOS2018, a united team from Fukuoka University and Osaka Prefecture University. The overview of the team's two recent research themes is also described. The first one is the method of online search of cooperative behavior and several approaches to acquire the appropriate evaluation functions. The second one is an opponent analysis in order to adopt the team strategy to the current opponent team online.

1 Introduction

This paper introduces the RoboCup 2018 Soccer Simulation 2D League champion team, HELIOS2018, a united team from Fukuoka University and Osaka Prefecture University. The team has been participating in the RoboCup competition since 2000, and won 2010, 2012 [1], 2017 and 2018 competitions. The team released several open source software for developing simulated soccer team using the RoboCup Soccer 2D simulator. A team base code, a visual debugger and a formation editor are available now. The details can be found in [3].

The reminder of this paper is organized as follows. Section 2 introduces the Soccer Simulation 2D League. Section 3 introduces our recent approach of online search of cooperative behavior. Section 4 introduces our approach of opponent analysis. Section 5 concludes.

2 Soccer Simulation 2D League

The RoboCup Soccer Simulation 2D League is the oldest of the RoboCup competition [9]. The simulation system[1] enables two teams of 11 autonomous player

[1] Available at: https://github.com/rcsoccersim.

© Springer Nature Switzerland AG 2019
D. Holz et al. (Eds.): RoboCup 2018, LNAI 11374, pp. 450–461, 2019.
https://doi.org/10.1007/978-3-030-27544-0_37

agents and an autonomous coach agent to play a game of soccer with realistic rules and game play. Figure 1 shows a screenshot of the 2D soccer simulator. Player agents receive a visual sensor message, an aural sensor message and a body sensor message from the simulation server, and can send few types of abstract action command (kick, dash, turn, turn_neck, and so on). In the game of RoboCup Soccer Simulation 2D League, player agents make a decision at each cycle in real time. A game consists of 6000 cycles, thus the decision making process of each player is executed about 6000 times. The 2D soccer simulator does not model the motion of any particular physical robot, but does capture realistic team level strategic interactions. Therefore, the performance of a team depends on the decision process of its agents. Due to its functions and stability, the 2D soccer simulator is known as a good research and educational tool for multiagent systems and artificial intelligence.

In 2018, up to 16 teams were allowed to pass the qualificaton process. Finally, 13 teams participated in the competition. HELIOS2018 won the championship with 18 wins and 1 loss and 1 draw, scoring 75 goals and conceding 9 goals. CYRUS2018 from Atomic Energy High School won the second place, and MT2018 from Hefei University won the third place.

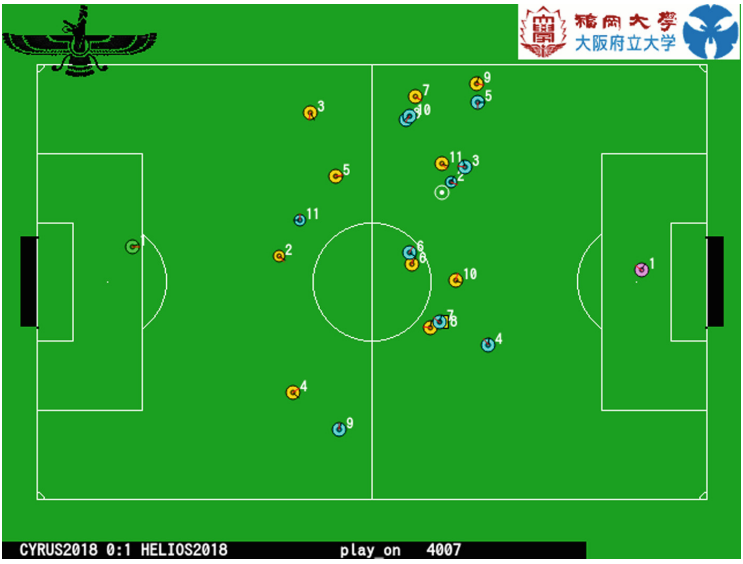


Fig. 1. A screenshot of the 2D soccer simulator. This game shows the RoboCup 2018 final between CYRUS2018 (Atomic Energy High School, Iran) and HELIOS2018 (Fukuoka University and Osaka Prefecture University, Japan).

3 Online Search of Cooperative Behavior

This section shows an overview of online search of cooperative behavior implemented in our team. First, the model of cooperative behavior and its planning process are described. Then, our recent approaches for acquiring the evaluation function are described.

3.1 Action Sequence Planning

In order to model the cooperative behavior among players as action sequence planning, we employed a tree search method for generating and evaluating action sequences performed by multiple players [2]. This method searches for the best sequence of ball kicking actions among several teammate players using a tree-structured candidate action generator and an evaluation function of the candidate actions. A lot of ball kicking action plans are generated during the search process and the best action plan is selected based on the evaluation value. In the current implementation, this planning process is only made by a ball kicker (called a kicker hereafter). First, the candidate actions are generated from the current situation of the soccer field. Each action is assigned an evaluation value that represents the quality of the action. The selected action is then used as a second kicking point to generate further candidate actions. This process expands the sequence of candidate actions in a tree form. Best first search algorithm is used to traverse a tree and to expand nodes. A path from the root node to the branch represents a kick action sequence that defines a certain cooperative behavior. We assume the considered actions are abstracted ones, such as pass, dribble and shoot. Figure 2 shows an example of kick action sequence. The example sequence starts with a pass from Player 10 to Player 7. Then Player 7 dribbles to a position where it passes the ball to Player 9, who finally makes a shoot. Since we consider only ball kicking actions, move actions such as ball interception are omitted in the figure. In our search process, the action plan is generated by the first ball kicker. In Fig. 2, Player 10 generates this plan, and other players cannot know the exact plan or can know it partially by using aural communication.

An example of the planning process is depicted in Fig. 3. The kicker generates three candidates for the first action (i.e., pass, pass, and dribble). Each of the three actions has an evaluation value in the corresponding node. That is, the evaluation value of the first pass is 30, the second pass is 20, and the dribble is 15. In this case, the first pass with the highest evaluation value is employed as the first action. Further candidate actions are generated from the selected pass action. We call the level of the tree as depth of the action sequence. Two actions (pass and dribble) in Depth 2 are added as the candidate action with the corresponding evaluation values. The action sequence is updated as the one with the highest evaluation value among the candidate. In this case, the pass in Depth 2 is selected as it has the highest evaluation of 35. Thus the resultant action sequence is “pass–pass”.

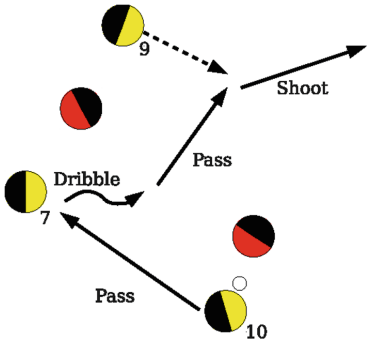


Fig. 2. An example of action sequence. This image shows the sequence of four actions: (1) pass from Player 10 to Player 7, (2) dribbling by Player 7, (3) pass from Player 7 to Player 9, and (4) Player 9 shoots to the goal.

The decision of players highly depends on the evaluation function, that computes the evaluation value of each action plan. We have to design an appropriate evaluation function in order to select the action plan corresponding to the team strategy and tactics. However, it is difficult to design an appropriate evaluation function because we have to consider many feature values.

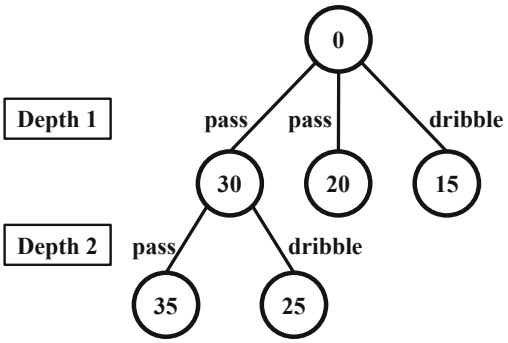


Fig. 3. An example search tree of action sequence planning.

3.2 Pruning in Action Sequence Planning

We use a pruning approach to restrict actions generated in tree search process. This approach enables to control players' action pattern without adjusting the evaluation function. We propose a pruning method using Support Vector Machine.

In order to clarify position transition of actions, two continuous variables, x and y that represent the coordinate value in the soccer field, are discretized. We

divide the soccer field into $n \times m$ grid. In the grid field, if the ball exists in a grid, the grid takes the value 1, otherwise the grid takes the value 0. This grid represents $n \times m$ dimensional vector.

In our approach, action sequences not intended by the team developer are pruned during tree search process. The pruning is determined by a classifier of Support Vector Machine. The labeling method to create a training data set is described in Fig. 4. After pruning process, only action sequences representing the tactics intended by the team developer remain. The input to SVM is a discretized coordinate value of position where kicking action is performed.

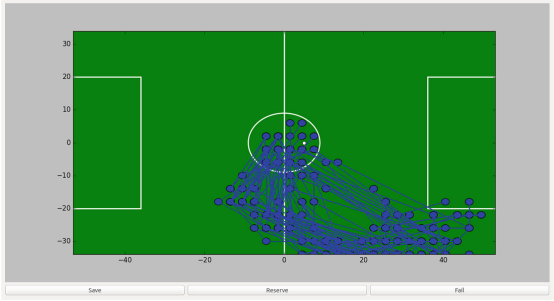


Fig. 4. A screenshot of the developed GUI for pruning.

Since SVM is a supervised learning method, we need a training data set to acquire classifier model. We extract action sequences from game log files. Then, the team developers set a label to the extracted action sequences using GUI if that is suitable for their intention. This labeled action sequence is used as a training data. We propose to apply a clustering method in order to classify similar action sequences. This approach reduces the human's action selection procedure. We use Gaussian mixture and EM algorithm with BIC as a clustering method. Figure 4 shows our GUI application. Action sequences organized into one cluster are displayed in the main window. Action sequences intended by the team developer are labeled "1", other action sequences are labeled "−1".

We performed an experiment to evaluate clasification performance of our SVM. Then, in order to evaluate the effectiveness of our pruning approach, we compare the perfomance of two teams, HELIOS2016 with the proposed model and the original HELIOS2016.

In order to evaluate our SVM classifier, we performed simulation games against 8 teams from RoboCup2016. 100 games were performed for each team. Then, extracted action sequences are classified by clustering algorithm. Classified action sequences are labeled by the team developer using our GUI application. Linear kernel and RBF kernel are used as the kernels of SVM. We applied 10-fold cross validation to the obtained training data set. We compared the two types of grid resolutions, 7×7 and 23×26. Table 1 shows the result of 7×7 and Table 2

Table 1. Accurasy rate of SVM (7×7)

Kernel	Depth: 1	Depth: 2	Depth: 3	Depth: 4
Linear	80.31	81.98	83.65	84.60
RBF	67.93	67.93	67.93	67.93

Table 2. Accurasy rate of SVM (23×26)

Kernel	Depth: 1	Depth: 2	Depth: 3	Depth: 4
Linear	83.44	85.08	86.58	87.45
RBF	67.93	67.93	67.93	67.93

shows the result of 23×26. These results shows Linear kernel is better than RBF kernel and the 23×26 grids is better than the 7×7 grids.

In order to evaluate our pruning approach, we performed simulation games against 8 teams. We used the classifier model with Linear kernel and 23×26 grids field. As a result, we find that HELIOS2016 with the proposed method has a stronger tendency to pass on one side compared with the original team, from the analysis of pass distribution.

3.3 Knowledge Sharing in Action Sequence Planning

The problem here is that even though the generated action sequence is good, its execution is not easy. Most of the times, the generated action sequence is intervened during the course of its execution and it is necessary to plan the next action sequence again. One reason for this is that non-ball-kicking players (we call this non-kickers hereafter) do not know the plan of the kicker, which leads the incomplete formation to perfectly execute the plan. In order to tackle this problem, knowledge sharing is employed. Knowledge sharing is to have a common knowledge among different players. In the context of the action sequence planning in this paper, the knowledge sharing means that the action sequence generated by non-kickers are made as close to the one planned by the kicker as possible. To do so, non-kickers focus on obtaining the information around the kicker as much as possible. Then they try to generate the same action sequence as that by the kicker. The non-kicker selects the next action considering the action sequence which will be executed from the current situation.

In order to examine the effectiveness of knowledge sharing, we have done a preliminary experiments using Team opuSCOM, which is based on agent2d. Knowledge sharing is implemented in the opuSCOM players so that all the players are able to expect the action sequence of a kicker when they are non-kickers. In particular, we focus on a situation depicted in Fig. 5. As shown in the figure, action sequences with the depth of less than or equal to two are only considered. Furthermore, it is assumed that all the actions in the plan are passes. This is because it is easier to measure the effectiveness of the knowledge sharing without considering the presence of dribbles, which are difficult to differentiate with self passes.

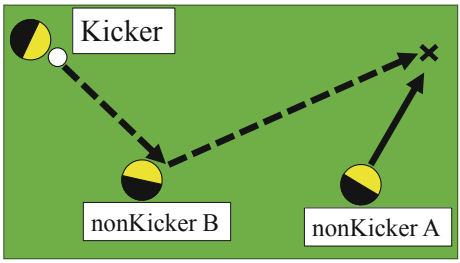


Fig. 5. Experimental situation.

Using ten teams that participated in the previous domestic as well as international competitions, the execution rates of the planned action sequences are measured when knowledge sharing is implemented in Team opuSCOM. First, we show the success rates of the knowledge sharing in Table 3. The table shows the average number of planned chain actions as well as successfully shared planned chain actions in 300 games for each opponent team. From this table, we can see that knowledge sharing is achieved with high precision.

Table 3. Success rates of the knowledge sharing.

Opponents	#Planning	Sharing rate (%)
agent2d	95	89.40
HillStone	89	89.64
Esperanza	84	90.16
Toyosu-Galaxy	30	92.20
WIT	51	89.49
Fifty-Storms	100	90.48
ITAndroids	84	89.81
PersianGulf2017	73	87.45
Ri-one2017	65	89.72
opuSCOM (Before)	114	90.29

Next, we investigate the execution of the planned action sequences. Table 4 shows the average number of planned action sequences (#Planning), the successful execution rates of the planned action sequences for the depth of one (D1) and two (D2), and the rate of successful execution of the depth-two action sequences over the successful depth-one action sequences (D2 per D1). Table 4 also compares the performance of the two versions of opuSCOM, one without the knowledge sharing (before) and the other with it (after). From this table, we can see that the successful execution increased by the introduction of knowledge sharing with the significance level of 5%.

Table 4. Successful execution rates of planned chain actions.

Opponents		#Planning	D1 (%)	D2 (%)	D2 per D1 (%)
agent2d	Before	101	73.37	42.12	57.40
	After	95	73.39	47.15	64.25
HillStone	Before	94	71.92	39.21	54.52
	After	89	70.20	45.79	65.23
Esperanza	Before	89	72.49	43.53	60.05
	After	84	73.34	48.92	66.70
Toyosu-Galaxy	Before	29	74.46	30.50	40.96
	After	30	75.41	37.12	49.22
WIT	Before	51	74.06	44.36	59.90
	After	51	73.57	47.56	64.65
FiftyStorms	Before	104	67.80	37.52	55.34
	After	100	68.73	43.01	62.58
ITAndroids	Before	91	73.63	35.61	48.36
	After	84	73.95	40.65	54.97
PersianGulf2017	Before	71	70.29	39.74	56.54
	After	73	68.67	40.25	58.61
Ri-one2017	Before	67	73.71	36.00	48.84
	After	65	73.06	37.03	50.68
opuSCOM (Before)	Before	120	71.13	41.04	57.70
	After	114	72.43	47.14	65.10

3.4 Learning Evaluation Functions by Supervised Learning Method

During the search process, all action sequences are scored by predefined evaluation functions. Usually, human developers manually design the evaluation function for their team. However, manual adjustment does not necessarily produce the optimal function and requires much trial-and error iterative operation. In order to acquire an appropriate evaluation function, we are trying to apply several supervised learning methods.

Learning to Rank algorithm is one of our approaches to acquire the evaluation function that reflects human's tactical intention. Learning to Rank is one of the machine learning techniques which is widely used in information retrieval domain. We applied a linear model of SVMRank [7,8] to acquire the evaluation function for action sequence planning. SVMRank is one of the methods of Learning to Rank, that is classified to the pairwise method. In order to create training data for SVMRank, human trainers are only required to choose preferred action sequences from all generated ones. We developed a viewer application that can visualize generated action sequences and enables us to choose them intuitively. We can observe the list of generated action sequences and their overview on the soccer field (Fig. 6). The details of this approach can be found in [4].

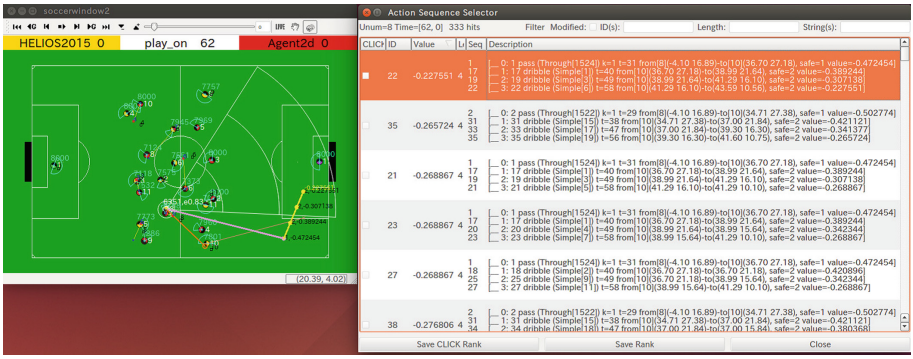


Fig. 6. Viewer application to choose preferred action sequences.

In another approach, we employed a neural network to model an expert team's evaluation function [6]. The neural network is trained by using positive and negative episodes of action sequences that are extracted from game logs. We used two versions of input features. One is the position at the next kick (x_n, y_n), which means that a two-dimensional input feature vector is used for training data. And the other input features are the position at the current kick and the ball position at the next kick ((x_c, y_c) and (x_n, y_n)). These features do not contain player positions.

The extracted episodes from log files are converted to generate training data for the learning of neural networks. For two-dimensional training data, the ball positions in an episode are separated into individual ball positions. Each of such ball positions is used as a training vector which consists of the ball position (x_n, y_n) as well as a positive/negative target value. This process is shown in the above side of Fig. 7. On the other hand, in the case of four-dimensional training data, a pair of successive two ball positions are used as a training vector. The former term of the pair is regarded as the current ball position and the latter is the predicted ball position at the next kick. Each of the two ball positions in the pair is concatenated to generate a four-dimensional input vector (x_c, y_c, x_n, y_n). The target value for the generated vector is determined by the label (i.e., positive or negative) that is associated with the episode that the four-dimensional vector was generated from. The lower part of Fig. 7 shows this process.

This neural network approach tries to mimic other strong team's strategy. However, we often need to modify a part of obtained strategy in order to affect team developer's preference. We are planning to integrate the idea of first approach to the second approach.

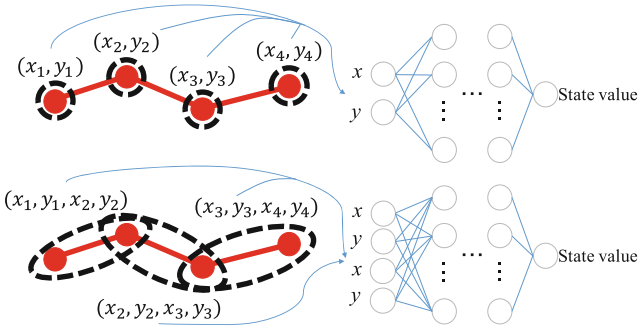


Fig. 7. Conversion of an episode into training data

4 Opponent Analysis

In this league, one of the essential tasks in the development of a team is to design an effective strategy. The sooner the opponent team's strategy is identified, the sooner the team can adapt its strategy in order to increase its chance to win the game. In this section, we introduce a method for identifying opponent teams' strategies.

We define a team strategy as the player's positioning during a game. To use players'position as the inputs of the learning model requires to consider their uniform numbers. Thus, the order of the players would be a problem in the construction of the model. To cope with this issue, an opponent formation is numerically expressed by discretizing the soccer field by a grid as shown in Fig. 8. Then, the number of players present in each cell is counted. The value of each cell is used as the input of the learning model. The value in each cell shows the number of opponent players at a certain cycle, and the results are integrated. Then, the average value is computed by dividing the integration obtained so far by the number of observed cycles. This set of the average values is used as input data of our identification model. For example, if the field is discretized by a grid of size 6×4, opponent formation is expressed by a 24-dimensional vector.

The input vectors are used to train a supervised-learning-based classification methods. SVM is employed as our identification model in order to classify the type of opponents teams' formations. The accuracy of the model is high even for short amount of time spent to analyze the opponent formation. The detailed results can be found in [5].

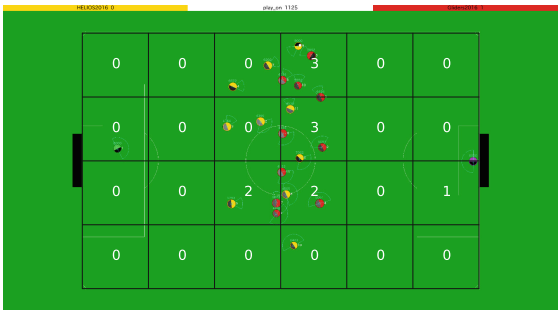


Fig. 8. Discretization of the soccer field by a grid of size 6×4.

5 Conclusion

This paper introduced the champion or RoboCup 2018 Soccer Simulation 2D league. First, the overview of the competition is described. Then, we described our current research topics, online search of cooperative behavior and opponent analysis. The HELIOS team won 3 championships in the past RoboCup competitions. Currently, our released software are widely used in the 2D community not only for the competition but for the research.

References

1. Akiyama, H., Nakashima, T.: HELIOS2012: RoboCup 2012 soccer simulation 2D league champion. In: Chen, X., Stone, P., Sucar, L.E., van der Zant, T. (eds.) RoboCup 2012. LNCS (LNAI), vol. 7500, pp. 13–19. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39250-4_2
2. Akiyama, H., Aramaki, S., Nakashima, T.: Online cooperative behavior planning using a tree search method in the RoboCup soccer simulation. In: Proceedings of 4th IEEE International Conference on Intelligent Networking and Collaborative Systems (INCoS), pp. 170–177 (2012)
3. Akiyama, H., Nakashima, T.: HELIOS base: an open source package for the RoboCup soccer 2D simulation. In: Behnke, S., Veloso, M., Visser, A., Xiong, R. (eds.) RoboCup 2013. LNCS (LNAI), vol. 8371, pp. 528–535. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-662-44468-9_46
4. Akiyama, H., Fukuyado, M., Gochou, T., Aramaki, S.: Learning evaluation function for RoboCup soccer simulation using humans' choice. In: Proceedings of SCIS & ISIS 2018 (2018)
5. Fukushima, T., Nakashima, T., Akiyama, H.: Online opponent formation identification based on position information. In: Akiyama, H., Obst, O., Sammut, C., Tonidandel, F. (eds.) RoboCup 2017. LNCS (LNAI), vol. 11175, pp. 241–251. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-00308-1_20
6. Fukushima, T., Nakashima, T., Akiyama, H.: Mimicking an expert team through the learning of evaluation functions from action sequences. In: Holz, D. et al. (eds.) RoboCup 2018. LNAI, vol. 11374, pp. xx–yy. Springer, Cham (2019)

7. Joachims, T.: Making large-scale support vector machine learning practical. In: Advances in Kernel Methods, pp. 169–184. MIT Press (1999)
8. Joachims, T.: Training linear SVMs in linear time. In: Proceedings of the ACM Conference on Knowledge Discovery and Data Mining (KDD) (2006)
9. Noda, I., Matsubara, H.: Soccer server and researches on multi-agent systems. In: Proceedings of IROS-96 Workshop on RoboCup, pp. 1–7 (1996)